

TIPE : Fonctionnement de la simulation de fluide

Thème : Cycles et Boucles

Sujet : Allées de tourbillon de Von Karman

1 Fonctionnement de objetcs.rs

Ce code fait partie d'un système de simulation de fluide en grille. Il permet de placer aléatoirement des objets solides (`walls`) dans la grille sous différentes formes : cercle, carré, triangle, rectangle.

1.1 Fonction principale : `place_random_objects`

But : Place un nombre donné (`num_objects`) d'objets aléatoires dans la grille.

Principe :

- Les positions sont choisies de manière à éviter les bords (via `border_margin`).
- Le type d'objet est tiré aléatoirement parmi 4 formes : `circle`, `square`, `triangle`, `rectangle`.
- Chaque objet est créé via la méthode correspondante, qui remplit certaines cellules avec l'état `wall = true`.

1.2 Formes

1.2.1 Square, Rectangle

Tracent un bloc rectangulaire autour d'un centre donné.

1.2.2 Triangle

- Trace un triangle équilatéral pointant vers le haut.
- La base s'élargit vers le bas.

1.2.3 Triangle_outline

- Trace uniquement le contour d'un triangle, sans le remplir.
- Utilise `draw_line` pour dessiner les segments reliant les sommets.

1.3 bresenham_line

But : Calcule une ligne discrète entre deux points donnés (x_0, y_0) et (x_1, y_1) sur une grille.

Principe :

- Utilise uniquement des entiers, ce qui le rend rapide et précis pour les maillages 2D.
- L'erreur de tracé est ajustée à chaque étape pour décider s'il faut avancer horizontalement, verticalement ou les deux.
- Retourne un `Vec< (usize, usize) >` contenant tous les points intermédiaires.

Détails techniques :

- dx/dy : décalages absolus en x et y .
- sx/sy : directions de progression (± 1).
- err : erreur cumulative qui guide le choix de la direction à chaque étape.
- $e2$: double de l'erreur actuelle, permet une comparaison sans division.

2 Fonctionnement de `pressure_computation.rs`

Ce code définit une série de méthodes associées à la struct `Grid`. Il permet de calculer les forces que le fluide exerce sur des objets solides (les “walls”), d’identifier les objets solides dans la grille, de regrouper les forces appliquées à chaque objet, et finalement d’afficher ces forces.

2.1 `compute_wall_forces`

But : Calculer les forces que le fluide exerce localement sur les cellules `wall` (objets solides).

Principe :

- Pour chaque cellule `wall`, on regarde ses voisines immédiates.
- Si une voisine n’est pas un mur, alors elle contient du fluide, et on considère qu’elle applique une force sur le mur.
- Trois types de forces sont modélisées :
 - Force de pression : $-p \times \text{normal}$
 - Force de traînée (*drag*) : proportionnelle à la vitesse normale du fluide
 - Force de cisaillement (*shear*) : proportionnelle à la vitesse tangentielle, pondérée par la viscosité

Le résultat est un vecteur `forces`, qui contient la force subie par chaque cellule `wall`.

2.2 `identify_objects`

But : Identifier les objets solides (constitués de cellules `wall` contiguës) et leur attribuer un identifiant unique (`object_id`).

Principe :

- On parcourt la grille. À chaque fois qu’on trouve un mur non encore étiqueté, on lance un DFS (par pile) pour explorer tout l’objet auquel il appartient.
- Chaque objet reçoit un `object_id` unique (1, 2, 3, ...).
- Le résultat est un vecteur `object_ids`, de la même taille que la grille, contenant l’ID de l’objet pour chaque cellule (ou 0 si ce n’est pas un mur).

2.3 `compute_object_forces`

But : Aggréger les forces calculées au niveau des cellules `wall` pour chaque objet entier.

Étapes :

1. Appel à `compute_wall_forces` → force appliquée à chaque cellule mur.
2. Appel à `identify_objects` → à quel objet appartient chaque mur.
3. Initialisation d’un tableau `objects` de struct `ObjectForce` pour stocker :
 - Force totale.
 - Centre de masse (approximé par les indices i, j).
 - Moment de force (*torque*).
4. Pour chaque cellule `wall`, on accumule :
 - Sa contribution au centre de masse.
 - La force qu’elle subit.
5. Ensuite, pour chaque objet, on calcule le moment de force (*torque*) par produit vectoriel entre :
 - Le vecteur centre de masse → cellule.
 - La force exercée sur l’objet.

2.4 `print_object_forces`

But : Affiche dans la console les résultats des calculs précédents.

Affiche pour chaque objet détecté :

- Nombre de cellules.
- Centre de masse.
- Force totale (et sa norme).
- Moment de force.
- Direction approximative (ex : ↑, ←...).

C’est une fonction de diagnostic ou de visualisation, utile pour savoir si des objets “flottent”, “tournent”, ou sont “poussés”.

3 Fonctionnement de condition.rs

Ce code définit un ensemble de constantes globales servant à paramétrer tous les aspects de la simulation de fluide, de l’affichage graphique à la dynamique du fluide, en passant par les sources, le maillage et le flux.

3.1 Paramètres de la fenêtre

- **ADAPT_TO_WINDOW** : active l’adaptation automatique de la taille de la fenêtre à la taille de la grille.
- **WINDOW_HEIGHT** / **WINDOW_WIDTH** : dimensions de la fenêtre, ajustées à la grille si **ADAPT_TO_WINDOW** est vrai, sinon fixées à 720×720 .

3.2 Paramètres de simulation

- **SIM_STEPS** : nombre de pas de temps à simuler.
- **PRINT_FORCES** : affiche ou non les forces calculées pendant la simulation.
- **CIP_CSL4** : active l’advection de densité avec le schéma CIP-CSL4 (non activé ici).
- **DENS_ADV_FAC** : facteur pour l’advection de densité.
- **VEL_STEP** : méthode choisie pour la mise à jour de la vitesse (“1”, “2” ou “cip_csl4”).
- **PROJECT** : méthode de projection utilisée (“1” ou “2”).
- **KARMAN_VORTEX** : tentative d’activer des vortex de Karman.
- **CENTER_SOURCE** : active une source de fluide circulaire au centre de la grille.
- **CENTER_SOURCE_TYPE** : **false** pour une sortie homogène, **true** pour un flux radial type ventilateur.
- **CENTER_SOURCE_RADIUS** : rayon de cette source.
- **CENTER_SOURCE_DENSITY** : densité injectée au centre.
- **CENTER_SOURCE_VELOCITY** : vitesse du fluide injecté par la source centrale.

3.3 Paramètres de flux

- **AIR_FLOW** : active un flux d’air global.
- **FLOW_DIRECTION** : direction du flux imposé (“left”, “right”, “up”, “down”).
- **FLOW_SPACE** : espacement entre les rangées de flux.
- **FLOW_DENSITY** : densité du fluide dans ce flux.
- **FLOW_VELOCITY** : vitesse de ce flux (0 si **AIR_FLOW** est désactivé).
- **INFLOW_VELOCITY** : forcée égale à **FLOW_VELOCITY** pour garder cohérence.
- **DRAW_VELOCITY_VECTORS** : dessine les vecteurs vitesse à l’écran.
- **VECTOR_SIZE_FACTOR** : facteur d’échelle des flèches de vitesse.
- **PAINT_VORTICITY** : colorie la vorticit   (rotation locale du fluide).

3.4 Paramètres de la grille

- **EXT_BORDER** : utilise des bordures   tendues (   garder **true** si activ  ).
- **N** : taille de la grille (**Attention : doit   tre (une puissance de 2) – 2** pour la compatibilit   avec l’encodage Morton).
- **DX, DY** : taille (en pixels) d’une cellule en x et y .
- **SIZE** : nombre total de cellules   quivalent    $(N + 2)^2$.

3.5 Param  tres physiques

- **DT** : pas de temps (en secondes).

3.6 Param  tres du fluide

- **VISCOSITY** : viscosit   du fluide. Une petite valeur favorise les tourbillons de type Karman.